

Multiclass Classification

One-vs-All, All-vs-All, micro- and macro-averaging, softmax with cross-entropy, and Bayesian classifiers

THE PROBLEM

From two classes to C classes

Training data $(x^{(i)}, y^{(i)})$, $i = 1, \dots, N$, with labels $y^{(i)} \in Y = \{1, 2, \dots, C\}$. A linear classifier $\text{sign}\langle w, \tilde{x} \rangle$ gives only two answers. Three ways out:

1 • Reduce to binary

One-vs-All, All-vs-All

Train several binary models and combine their answers.

2 • One multinomial model

Softmax regression

C linear scores turned into one probability vector.

3 • Generative model

Bayes, Naive Bayes

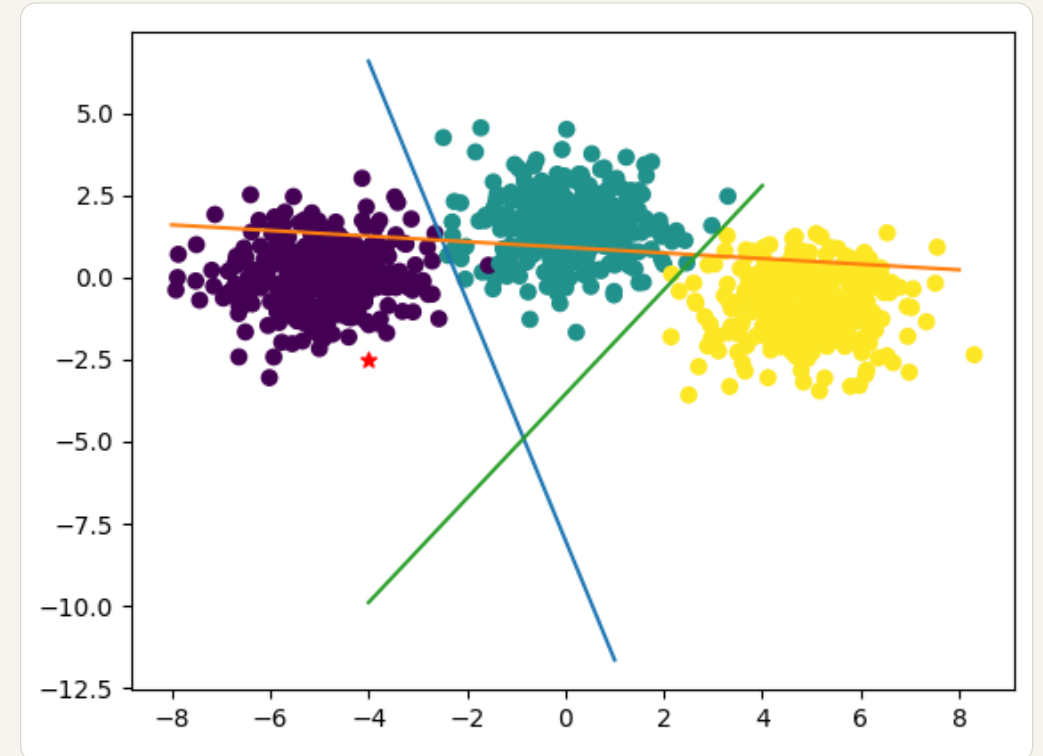
Estimate $p(x | y)$ and $p(y)$, then apply Bayes rule.

8.1.1 ONE-VS-ALL

One-vs-All: C models, most confident wins

1. For each class $k = 1, \dots, C$ relabel the data: positive if $y^{(i)} = k$, negative otherwise.
2. Train a binary model a_k on $(x^{(i)}, [y^{(i)} = k])$.
3. $a_k(x)$ returns a score: the signed distance $\langle w_k, \tilde{x} \rangle$ or a probability.

$$a(x) = \operatorname{argmax}_{c=1,2,\dots,C} a_c(x).$$



Notebook: three OvA lines; red star at $(-4, -2.5)$

EXAMPLE · POINT (-4, -2.5)

Three probabilities, one argmax

Model	Trained to detect	P(own class x)
clf1	class 0 vs rest	0.99962
clf2	class 1 vs rest	0.00038
clf3	class 2 vs rest	0.0000021

Prediction: argmax $\rightarrow$ class 0

Each model is fitted on its own. Their scores are not one distribution over classes and are not forced to sum to 1: this motivates softmax (8.3).

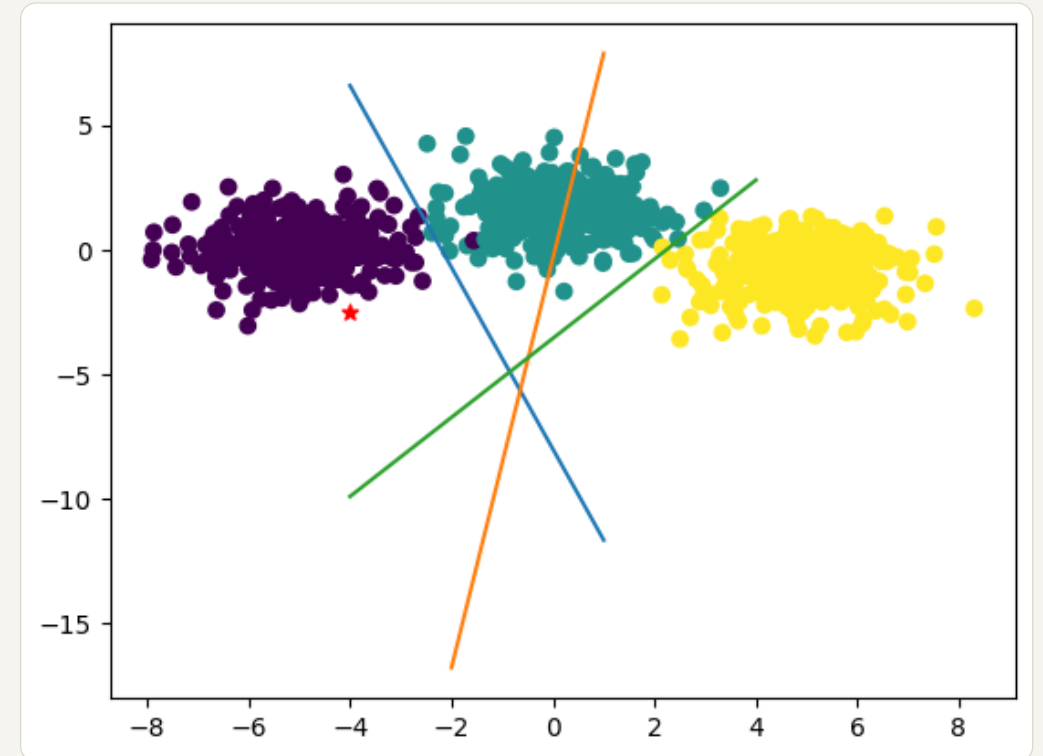
8.1.2 ALL-VS-ALL

All-vs-All: one model per pair, then vote

1. For each pair of classes $k < m$ keep only the points with $y \in \{k, m\}$.
2. Train a_{km} on them; it answers k or m .
3. Every model casts one vote; the class with most votes wins.

$$a(x) = \operatorname{argmax}_{c_1=1,2,\dots,C} \sum_{c_2=1}^C [a_{c_1,c_2}(x) = c_1].$$

$C(C-1)/2$ models: 3 for $C=3$, 45 for $C=10$.



Notebook: three pairwise lines, same red star

EXAMPLE · POINT (-4, -2.5)

Counting the votes

Pairwise models

Model	Pair	Vote
clf1	0 vs 1	0
clf2	0 vs 2	0
clf3	1 vs 2	1

Vote matrix [$a_{km}(x) = k$]

k \ m	0	1	2	Votes
0	.	1	1	2
1	0	.	1	1
2	0	0	.	0

Prediction: class 0 with 2 votes. Ties are possible (e.g. 1–1–1 with $C = 3$) and need a rule, such as summed confidences.

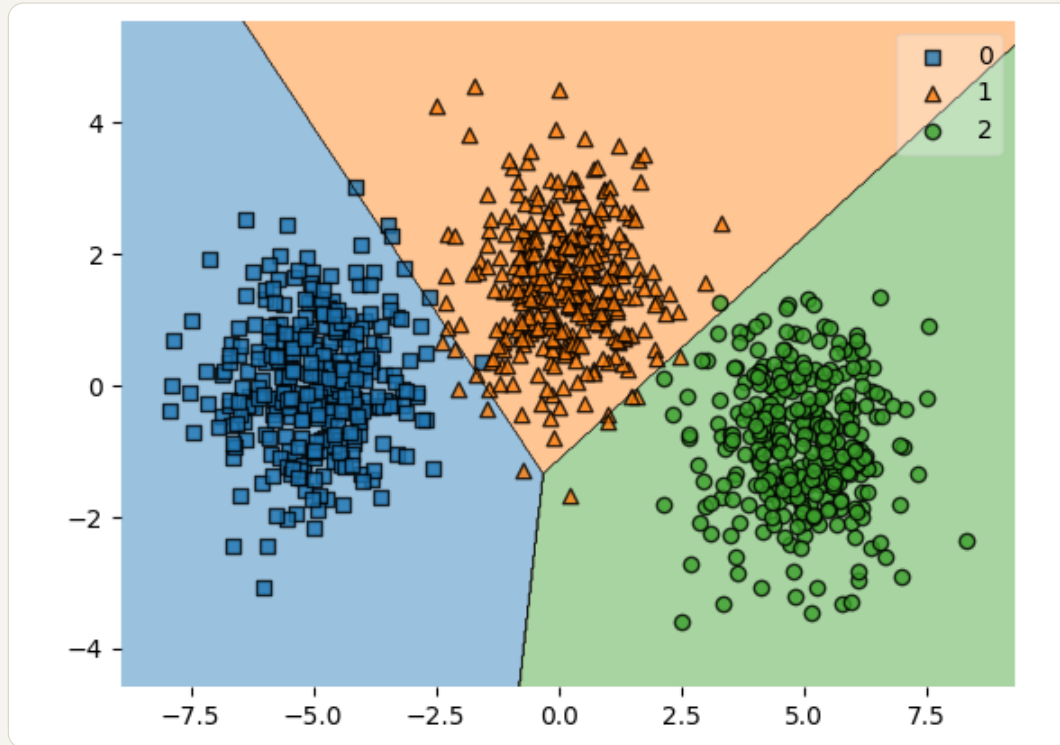
OVA VS AVA

Fewer models or easier problems

	One-vs-All	All-vs-All
Models	C	$C(C - 1)/2$
Data per model	all N points	two classes only
Class balance	1 class vs $C - 1$: skewed	pairs: closer to balanced
Each subproblem	can be hard (middle blob)	usually easier
Combining	compare scores; need one scale	count votes; ties possible
scikit-learn	OneVsRestClassifier	OneVsOneClassifier

DECISION REGIONS

argmax of linear scores cuts the plane into convex cells



`LogisticRegression(multi_class='ovr')` fits all three OvA models in one call; `mlxtend`'s `plot_decision_regions` colors each point of the plane by $a(x)$.

Region k is where $a_k(x) \geq a_m(x)$ for every m : an intersection of half-planes, so it is convex.

8.2 QUALITY METRICS

Micro vs macro: two ways to average

Accuracy extends directly:

$$Accuracy = \frac{1}{N} \sum_{i=1}^N \left[a(x^{(i)}) = y^{(i)} \right]$$

Precision and recall are per class: treat class k as positive to get TP_k , FP_k , FN_k , TN_k .

Micro: pool counts first

$$TP = \sum_{c=1}^C TP_c, \dots$$
$$Precision = \frac{TP}{TP + FP}$$

Dominated by the largest class.

Macro: per class, then average

$$Precision_c = \frac{TP_c}{TP_c + FP_c}$$
$$Precision = \frac{1}{C} \sum_{c=1}^C Precision_c$$

Every class counts equally, whatever its size.

EXAMPLE · ILLUSTRATIVE COUNTS

One big class hides two weak ones

Class	TP_k	FP_k	$Precision_k$
A	90	10	0.90
B	5	5	0.50
C	4	1	0.80
Total	99	16	

Micro

$$99 / 115 \approx 0.86$$

Macro

$$(0.90 + 0.50 + 0.80) / 3 \approx 0.73$$

8.3 MULTINOMIAL LOGISTIC REGRESSION

Softmax: C scores become one distribution

$$a_k(x) = \langle w_k, \tilde{x} \rangle, \quad k = 1, \dots, C$$

$$p_{w_c}(x) = \frac{e^{a_c(x)}}{\sum_{k=1}^C e^{a_k(x)}}$$

- $p_k > 0$ and $\sum_k p_k = 1$
- keeps the order: $\operatorname{argmax} p_k = \operatorname{argmax} a_k$
- adding one constant to all a_k changes nothing

Scores (2, 1, -1)

k	a_k	$\exp a_k$	p_k
1	2	7.389	0.705
2	1	2.718	0.259
3	-1	0.368	0.035
Σ		10.475	1

8.3 TRAINING

Maximum likelihood keeps only the true class

$$L(y^{(1)}|x^{(1)}, \dots, y^{(N)}|x^{(N)}; w_1, \dots, w_C) = \prod_{i=1}^N p_{w_1}(x^{(i)})^{[y^{(i)}=1]} \cdot \dots \cdot p_{w_C}(x^{(i)})^{[y^{(i)}=C]}$$

$$\log L = \sum_1^N \left([y^{(i)} = 1] \log p_{w_1}(x^{(i)}) + [y^{(i)} = 2] \log p_{w_2}(x^{(i)}) + \dots + [y^{(i)} = C] \log p_{w_C}(x^{(i)}) \right).$$

For each point only one indicator $[y^{(i)} = k]$ equals 1, so the inner sum picks the log-probability the model gives to the **correct** class. Probability spent on wrong classes is lost from it.

$$\log L = \sum_{i=1}^N \sum_{c=1}^C [y^{(i)} = c] \log p_{w_c}(x^{(i)}) = \sum_{i=1}^N \log p_{w_{y^{(i)}}}(x^{(i)}) \rightarrow \max_{w_1, \dots, w_C}.$$

8.3.1 CROSS-ENTROPY LOSS

Negative log-likelihood is cross-entropy

$$Loss = -\frac{1}{N} \log L = -\sum_{i=1}^N \left([y^{(i)} = 1] \log p_{w_1}(x^{(i)}) + [y^{(i)} = 2] \log p_{w_2}(x^{(i)}) + \dots + [y^{(i)} = C] \log p_{w_C}(x^{(i)}) \right).$$

$$H(p, q) = -E_p[\log q] = -\sum_{j=1}^J p_j \log q_j, = H(p) + KL(p \parallel q)$$

Per point: p = one-hot true label, q = predicted ($p_1(x)$, ..., $p_C(x)$). The loss is smallest when $q = p$, since $KL(p \parallel q) \geq 0$.

Prediction for true class	p	$-\log p$
confident, right	0.705	0.35
confident, wrong	0.035	3.35

8.4 BAYESIAN CLASSIFICATION

Bayes rule turns class models into a classifier

$$p(Y = 1|X = x) = \frac{p(X = x|Y = 1)p(Y = 1)}{p(x|Y = 1)p(Y = 1) + p(x|Y = 2)p(Y = 2) + \dots + p(x|Y = C)p(Y = C)}$$

Prior $p(y)$

How common each class is before seeing x .

Posterior $p(y | x)$

Updated belief after seeing x .

MAP decision

$$\begin{aligned} a(x) &= \operatorname{argmax}_c p(Y = c|X = x) \\ &= \operatorname{argmax}_c p(X = x|Y = c)p(Y = c). \end{aligned}$$

8.4.1 NAIVE BAYES

Assume features are independent given the class

$$p(X_1, \dots, X_d | Y) = p(X_1 | Y) \cdot \dots \cdot p(X_d | Y).$$

$$p(w|ham) = \frac{\text{\#of times word } w \text{ used in ham}}{\text{total \# of words in ham}}$$

$$p(w|ham) = \frac{\text{\#of times word } w \text{ used in ham} + \delta}{\text{total \# of words in ham} + \delta|V|}.$$

$$p(text|ham) = \prod_{w \in text} p(w|ham)^{N(w, text)}, \text{ where } N(w, text) \text{ is the count of the word } w \text{ in the text.}$$

$$p(ham|text) \propto p(ham) \prod_{w \in text} p(w|ham)^{N(w, text)}$$

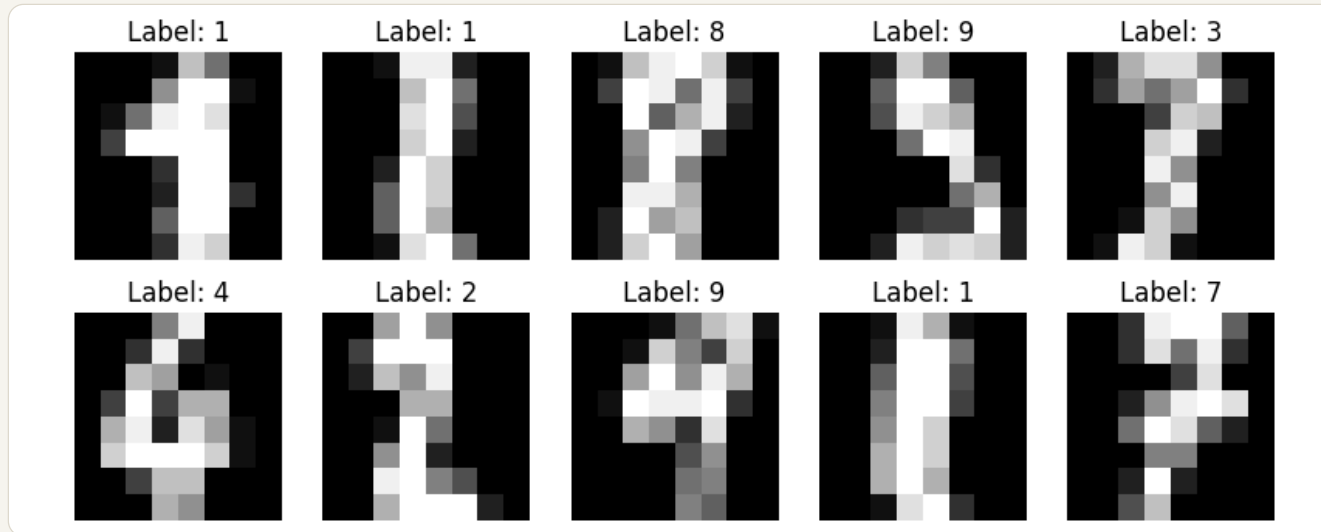
Spam filter, in logs

$$\log p(ham|text) \propto \log p(ham) + \sum_{w \in text} N(w, text) \log p(w|ham).$$

Logs avoid underflow from multiplying many small numbers.

NOTEBOOK · HANDWRITTEN DIGITS

Ten classes, 64 pixels, one softmax model



1203 / 594

train / test images, $8 \times 8 = 64$ features

10×64

coef_: one weight vector per class

0.97

test accuracy; macro F1 also 0.97

RECAP

Four ways to go multiclass

Strategy	What is trained	How x gets a class
One-vs-All	C binary models	argmax of the C scores
All-vs-All	$C(C - 1)/2$ binary models	majority vote over pairs
Softmax	C weight vectors jointly	distribution $p_k(x)$; cross-entropy loss
Naive Bayes	$p(y)$ and $p(x_j y)$	MAP: argmax of prior $\times$ likelihood

Evaluate with accuracy plus micro and macro averages; they disagree exactly when classes are imbalanced.